

College of Engineering
COMP 491 – Computer Engineering Design
Project Proposal

CENTRALIZED FEDERATED LEARNING FRAMEWORK

Fall 2026

Participant Information:

Name	ID	Email	Phone
Bora Toker	84968	btoker23@ku.edu.tr	05378669090
Enes Tunç	83357	etunc22@ku.edu.tr	05354412373
Hilal Aygül	83701	haygul22@ku.edu.tr	05467464395
Merve Köybaşı	79169	mkoybasi21@ku.edu.tr	05457956184

Project Advisor: Mehmet Emre Gürsoy, Barış Akgün

Abstract

Federated Learning (FL) is a technology for training machine learning models based on data that cannot be shared, such as data from financial institutions or health care providers, and telecommunication centers. Each organization keeps its raw data and is jointly responsible for creating the joint model in FL. Most of the current literature focuses on simulations on a single machine. But it may not be enough for the use of federated learning among organizations [1]. Federated learning on a single machine is quite different from federated learning between two independent organisations for several reasons. For instance, in federated learning among independent organizations, every organization acts independently, thus if one organization has incompatible data, then the action of that organization will affect the overall global model [1]. Other issues are slow clients, connection dropouts and duplicate updates, which are all a huge problem when it comes to training. It can be challenging to reconstruct the previous training state in case anything goes wrong. The main difficulty in training federated learning in actual production environments is handling distributed systems' inherent challenges, including but not limited to concurrency control, communication failures, synchronization problems, consistency requirements, security threats, and scalability limitations.

This paper introduces our cross-silo FL framework designed to solve the aforementioned problems. Our framework involves the central controller component, which oversees all clients' activities. The controller manages all independently functioning clients and deals with late, dropped out, or returning participants by excluding incompatible participants and letting only compatible ones engage in training. After every round of experiments, logging is performed for all trials through reports, graphs, or online web applications showing current results in real time. Finally, a trained model is delivered to all participant clients. Experiments are conducted on tabular banking datasets, and one additional modality (image or text) in simulation and in reality among different machines over the Internet. As contributions, we seek an implementable framework along with publicly available source code, step-by-step instructions for running our experiments, demonstrations, and the technical report describing the findings. It is crucial to emphasize that our study focuses on creating a cross-silo FL framework rather than developing new algorithms for learning.

TABLE OF CONTENTS

SECTION 1 4

1.1 4

1.2 4

1.3 6

SECTION 2 8

2.1 8

2.2 9

2.3 18

2.4 19

2.5 20

2.6 20

SECTION 3 20

SECTION 4 21

Section 1 Introduction

1.1 Concept

Federated Learning (FL) enables various companies to jointly train a model without exposing their data. This approach works for financial institutions and hospitals: there could be multiple reasons why it's not possible to use all of their data together. There are various FL approaches, such as FedAvg and FedProx algorithms. The Flower and NVIDIA FLARE frameworks are intended for making it easier to federate models. If the model is federated between many participants, a lot of problems arise at the system level such as: compatibility of the clients, instability of the network, instability of the clients, etc., which can't be addressed only with algorithmic research. For example, when considering the problem of incompatible clients, this means that while the clients have different order or scales of the representation of their features, or other features in their respective datasets, no errors occur during training which will ultimately hurt the global model.

To meet these challenges, we propose a framework using Federated Learning in this research paper. It is a cross-silo type framework that allows clients to operate autonomously, and data to be exchanged via the gateway channel. The goals of this FL framework are to provide training for the FedAvg and FedProx algorithms using simulation and a number of machines in the Internet, providing continuity even if some of the clients join late or they are not able to participate in training training for various reasons such as compatibility issues or duplicated updates. In addition, a web application should be used to log all of the steps in a training process.

Design-first incrementality is the foundation of our research methodology. Once the architecture and communication protocols were established, we implemented the framework incrementally by adding new components one layer at a time. Specifically, we built the FL core simulation environment, a FastAPI gateway with TLS-encrypted communication and a PostgreSQL database,the orchestration layer with a state machine for handling rounds, checking compatibility based on conformance contracts and client versions, logging and monitoring functionalities, and finally, a NextJS frontend website with Grafana dashboards. Each component will be evaluated independently via fault injection, incompatible client simulations, and multi-node deployments with actual experiments. We will compare FedAvg and FedProx on a tabular dataset and one image or text dataset.

In conclusion, successful outcomes would be obtaining a functioning system that is able to manage the interactions between different sites, exclude incompatible clients, withstand realistic network conditions and failures, and provide understanding of the learning process and system's elements' performance. The deliverables would be comprised of the following items: open source codebase, website with instructions on execution, video demonstration of functionality, poster presentation, and final report.

1.2 Objectives

The principal aim of this project is to develop a functional and reusable cross-silo Federated Learning (FL) framework until January 15th, 2027, where the central coordinator supervises independent client entities. Federated Learning enables corporations such as banks to jointly learn without transferring raw datasets among each other. However, the majority of current research remains limited to simulations. Our framework is designed to support practical implementations, allowing collaborating corporations with inaccessible databases to jointly learn while having transparency on the process by those operating a particular iteration of the system. This paper was conducted for the use of participating organizations. Our project will be

evaluated by our project advisors, course coordinator, and TAs, as well as faculty and company advisors at the poster presentation session.

In this project, the adopted methodology has been design-first. The design part included architecture definition, state machine definition, round definitions, protocol descriptions, database schema definition, and contract compliance checks. In this initial phase lasting two weeks, the bank datasets were prepared for our experiments. After finishing the design process, we started working on implementing essential components such as the coordinator and clients. Our implementation of components trains neural networks on local simulation using the federated learning algorithms – FedAvg and FedProx with the help of PyTorch framework. Our system supports partial participation and preliminary experimental results will be obtained on the tabular dataset.Components include a gateway built with FastAPI that receives the updated weights from registered clients over HTTPS with TLS. Model weights updates received from clients were saved to disk file storage. Every weight update was recorded to PostgreSQL database. Coordinator, developed by us, executes every state machine round sequentially and deals with minimum participation rate, timeouts, retries, stragglers, disconnected participants, obsolete and duplicate updates. Should interruption occur during the execution process, then it continues from where the last successful round ended. Utilizing our common transportation interface, components were run in distributed environment between multiple machines across the internet network.

The verification stage of the client should take place prior to the joining of the run, as semantic discrepancies between clients may damage the global model. In the case of managed-code clients, the version numbering will rely on the version number of the containers/bundles. As for independent clients, they must adhere to the conformance agreement as well as passing all supplied tests. After completion of a run, publication of the trained global model occurs in the form of a versioned artifact accompanied by a checksummed file. Authenticated clients can obtain this artifact to verify authenticity.

Every completed run produces a machine-readable log and a concise human-readable report from which the run can be fully reconstructed without inspecting console output. Coordinator saves the identification numbers, durations, latencies, message lengths, and dropout amounts for every round in PostgreSQL. After conducting all simulation runs, the system will perform automatic exporting of results in JSON/CSV formats, generation of Markdown reports, and creation of plots. The presentation layer consists of a Next.js framework web application. The public space is responsible for serving the project website, where updates are provided throughout the entire duration of the semester starting from its beginning. Meanwhile, the private space includes different modules such as Round Viewer, Update Inbox, Loggings, Reportings, Plotting, Client Compatibility, and Grafana Dashboard.

Empirical evaluation of our study includes experimentation with two types of datasets: a tabular dataset and one image or text dataset. We compare the FL algorithms FedAvg and FedProx, and federated training with two baselines: centralized learning and local-only learning. Performance metrics for the global model and each client's model are measured in addition to communication costs and runtime statistics. Some of the experiments are repeated on actual deployments and in simulations where failure modes are injected. Every experiment conducted is completely reproducible from our saved configuration file, random seeds, and version control. Under certain conditions when sufficient time allows, we conduct experiments involving privacy analysis as well.

The assignment concludes with the submission of the poster, demo video, clean repository, and project report. Project completion means completing all the above requirements apart from implementing the optional privacy extension module. All deliverables must function correctly over the Internet on different machines when submitted along with the report.

1.3 Background

State of the art:

Federated Learning refers to the training process of a shared machine learning model across different parties where none of the initial data transfer occurs to a central server. Federated Learning and its FedAvg algorithm were first suggested by McMahan et al. [1]. Clients update the current global model by training the model multiple times locally on their datasets, and then the server averages out all updates made by clients to aggregate parameters [1].The authors showed empirically that the approach works even when the data is unbalanced and non-IID across clients, and that it needs 10 to 100 times fewer communication rounds than synchronized SGD. Furthermore, too many local epochs may stall or even drive the system toward divergence [1]. To address these issues, FedProx [2] expands upon the idea of FedAvg by introducing a proximal term to the client's loss function and providing flexibility in computations to the clients based on their computational capabilities [2].

As described in their paper, Kairouz et al. [3] present an overview of the field and divide Federated Learning (FL) into two categories depending on the setting. Cross device FL takes place when many unreliable devices join the training process, whereas cross silo FL happens when only few trustworthy devices join training since law prevents the exchange of data among entities [3]. For instance, in the field of finance, cross-silo Federated Learning may be utilized for predicting risks. According to Rieke et al. [4], Federated Learning provides healthcare organizations with the opportunity to build joint predictive models without exchanging data sets by resolving privacy and security challenges, heterogeneity of data, traceability difficulties, and architectural challenges.

There are several open source frameworks that have implemented these techniques. Flower [5] is an open-source framework designed specifically for large scale federated learning experiments on heterogeneous devices, and to port them onto real devices. The majority of the existing frameworks offer FL algorithm simulations only [5] according to the creators of FLOWER. Further, the recent survey paper [3] mentions simulation libraries as one of the popular tools for FL studies and thus the trend is supported. NVIDIA FLARE is an open source software development kit specifically developed to support FL algorithms [6]. This framework aims in particular to implement federated learning between institutions across institutions [6]. This toolkit works with popular frameworks such as PyTorch, TensorFlow, XGBoost and NumPy. The toolkit not only provides secure communication facilities such as Mutual TLS Authentication, Provisioning Service and Role Based Access Control but also supports efficient management of the entire FL process [6]. Other libraries are available such as FedML with a focus on benchmarking FL algorithms for education purposes, and TensorFlow Federated for decentralized ML studies and simulations [7] and [8].

These methods can help greatly reduce implementation cost, and some can address challenges associated with securing deployment. Despite examining literature related to these techniques [5]-[7], there seems to be no solution proposing an approach to verifying whether connected nodes possess compatible semantic understanding. This paper attempts to fill in this knowledge gap besides addressing fault tolerance orchestration and observability problems.

Problem and its importance:

Institutional bodies keep personal information, particularly healthcare facilities, banks, firms with customer relations, and many others. Data protection laws do not forbid central storage as such, but they strictly limit how personal data may be collected and processed [9], [10]. The management of personal data regulations happens with the help of legal regulations concerning data protection. Those legal regulations include rules related to processing activities of personal data [9], [10]. GDPR law states that personal data must be appropriate, relevant, and minimally sufficient for intended purposes [9]. Similarly, KVKK states that personal data processing activities must be relevant and proportional to their intended objectives [10].Centralizing patient data involves numerous regulatory obstacles related to the protection of privacy.

Centralized datasets will certainly attract hackers because of their astronomical value worth tens of millions. According to the 2026 edition of IBM Cost of a Data Breach report, there was a 12% rise in the total expense caused by the data breach to $4.99 million globally. Amongst industries, healthcare experienced the highest total cost per data breach incident amounting to $6.64 million. Following the healthcare sector, finance followed at $6.29 million per incident, and a breach took 247 days on average to identify and contain [11].. Unlike the data pooling approach, federated learning overcomes this challenge via localized data [1],[3].

Moving from simulation to a real cross-silo deployment, however, raises problems that algorithmic studies largely abstract away:

- **Incompatible clients.** Independent sites may carry out pre-processing steps differently, and therefore the formatting of data poses a significant issue in federated learning among different organizations [4]. Even the presence of a single client whose data features are arranged differently, measured in different units (days/months), differently encoded categorical data, and labels having different meanings will ruin the whole model, although tensors might be correctly shaped.
- **Unreliable operation.** Cross-silo clients outperform cross-device clients concerning reliability [3]. However, potential issues remain in both architectures. Such issues include but are not limited to slow clients, connection failures, repeated updates, and restarting coordinators [3]. Moreover, client devices' processing capabilities vary [2], [5].
- **Limited observability.** In order to deploy multi-institutional models, there must be traceability mechanisms in place [4]. The proposed framework offers an auditing feature that helps to meet this requirement [6]. Displaying model execution will be impossible without proper logging, metric, and reporting features.

Recently, due to the developments of Federated Learning (FL), this topic has gained traction and is moving from experimental FL solutions to actual deployment in enterprise context. Cross silo FL applications are one of the possible applications of FL use case scenarios across silos, with current approaches attempting to facilitate this cross silo FL applications [6]. In scenarios across silos, apart from model performance, there are new challenges that must be addressed in terms of client onboarding safety, system stability and audibility concerns. Unlike previous works which only considers designing new algorithms for FL, we consider addressing these concerns first in our work.

Section 2 S/T methodology and associated work plan

2.1 Methodology

2.1.1 Overall approach

Design-first and iterative procedures have been adopted for the implementation of this research project. Framework designing involves a period of two weeks, whereby architecture, round state machine, protocols, conformity agreements, and database schema design have been carried out. Through this method, all team members will be working on the same specifications. Following this phase, our implementation and testing are done layer-wise, beginning with FL-core simulations, then secure communication and orchestration implementations, fault and crash implementations, and finally moving towards multi-node deployments, compatibility tests, and experiments. The web UI layer creation is done based on information obtained from previous layers' implementation and testing phases. FedAvg and FedProx algorithms were implemented through PyTorch.

2.1.2 Architecture and round protocol

Training occurs locally, and no data gets uploaded as the framework does not involve itself in the training step. Instead, only the updates are collected through a FastAPI gateway. Clients always initiate the connection, and all messages are encrypted with TLS. Once connected, the updates get stored in files, and logs are added to the database. There exists a coordinator created by the authors using Python, which controls the rounds of the state machine. The state machine has five different states, OPEN, COLLECTING, AGGREGATING, and either DONE or FAILED. Coordinator collects logs of updates, determining the validity by ignoring stale updates and duplicated uploads. When the necessary number of participants joins or timeouts occur, aggregation takes place. All state changes regarding rounds and model states are kept after each round completion to ensure resuming the state machine whenever the coordinator shuts down. Clients and coordinators communicate via a transport interface with two implementations – in-memory and HTTPS. We will take measures to make it harder for a client to infer other clients' data from the global model and its own weights.

2.1.3 Client compatibility

A client with different ordering of the features, units, encoding scheme, or interpretation of labels may inadvertently corrupt the global model since the tensors will fit without raising errors despite having different meanings. Prior to participating in the federated learning run, the client provides metadata concerning its model architecture, feature definitions, preprocessing pipelines, and label semantics to be checked against the run contract by the coordinator. Furthermore, the coordinator supplies reference input examples together with the corresponding expected outputs to validate the output of the preprocessing pipeline of the client. If the client fails, then it will be automatically disqualified with logging the failure reason. In managed-code clients, both containers and releases are always synchronized.

2.1.4 Validation

Each behavior will be assessed through direct testing, which encompasses fault injection wherein client delays/abort requests, duplicates and provides outdated updates, and kills the coordinator during execution. Client incompatibility and execution over the internet between multiple machines will also be investigated. Experiments will compare FedAvg against FedProx, and federated learning against centralized learning/local learning for two data modalities (one of them will be tabular, one of them will be image or text dataset).

2.1.5 Relevance, effectiveness, and novelty

Relevance: The solution is designed to address the issues emerging with applying federated learning in practice within enterprises after simulation, including client compatibility, unstable networking, and poor traceability.

Effectiveness: For every objective in this project, there is a mechanism designed, along with testing for its effectiveness. In order to maximize our resources efficiently, existing components such as PostgreSQL, Grafana, and TLS are used whenever possible.

Novelty: Previous platforms provide the functionality of algorithms, simulation, and safe deployment, but none of the research papers reviewed contain data semantics validation between independently developed client modules prior to joining. Our proposed solution includes three contributions: the data semantics checker, the orchestration layer created by the team, and a reconstructable log.

2.2 Work Package Descriptions

Work package number	**1**	Start date or starting event:		**Week 1-Week 2**	
Work package title	Design, plan and data preparation				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**2**	2	2	2	

Objectives

- Fix, in writing and with diagrams, exactly what the platform will do and how it will be built.
- Decide which features are mandatory and which are optional, and the order in which they are built.
- Plan the project web site and set up the team's repository and task tracking.
- Obtain the banking dataset from the advisor, confirm its source and license, and prepare it so that the first training run

Description of work

T1.1 (w1) Requirements and scope decision. Review the capstone project description and the advisor's input. Decide the mandatory and optional features, the pages of the web application, and the build order.

T1.2 (w1-w2) Architecture and protocol design. Draw the architecture diagram (coordinator, API gateway, clients, PostgreSQL, file storage, Grafana and web application), draw the round state machine (OPEN, COLLECTING, AGGREGATING, DONE or FAILED) draw the sequence diagrams (registration, a normal round, dropout and reconnection, coordinator crash and resume), and draw the deployment diagram for simulation of system and real multi machine setups.

T1.3 (w1-w2) Interface, database and conformance design. Specify API and message formats, database schema, and the conformance contract (what checks are done, in what order, and what is logged on rejection).

T1.4 (w1-w2) Data acquisition and preparation. Get the tabular banking dataset, checking its licensing and terms of use, explore, clean and preprocess dataset, and create a fully randomized (IID) partition across clients.

T1.5 (w2) Web site planning. Define structure of the project web site: public area (project info, documentation and results) and the authenticated area (live round view, logs, reports, plots).

T1.6 (w1-w2) Work plan and team organization. Create GitHub repository and issue tracker and also break the project down into tasks and assign owners. Create the weekly schedule and milestones.

Deliverables

D1.1 First progress meeting will be done with presentation of draft architecture and project plan.

D1.2 Design package: architecture diagram, round state machine, sequence diagrams and deployment diagram will be shown.

D1.3 API and message specification, database schema, and conformance contract specification.

D1.4 Scope decision and work plan (task breakdown, owners, weekly schedule).

D1.5 Prepared banking dataset with its IID partition and a note on its source and license.

D1.6 Plan for the project web site.

Milestones

M1.1 Repository and issue tracker are ready, and the draft design is presented at the first progress meeting.

M1.2 Design and scope are finalized.

M1.3 Dataset is ready for the first training run.

Work package number	**2**	Start date or starting event:		**Week 3-Week 5**	
Work package title	Core FL in simulation mode				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**3**	3	3	3	

Objectives

- Build a coordinator and clients that train PyTorch neural networks with FedAvg and FedProx in local simulation mode.
- Define a transport interface with an in-memory implementation, so that the same training and orchestration code can later run over HTTPS.
- Support client registration, model and configuration distribution, local training, update collection, aggregation, evaluation, and partial participation.
- Produce early results on the tabular banking dataset (federated, centralized, and local-only baselines) and tune the model.

Description of work

T2.1 (w3) Project skeleton and transport interface. Set up the code structure, configuration, and run script. Define transport interface in between coordinator and clients. Implement it in memory.

T2.2 (w3) Loading data and local training. Load the prepared client partitions from WP1 Build the Pytorch model for tabular data Implement local training and evaluation.

T2.3 (w3) Coordinator and FedAvg. Implement client registration, the round loop, model and configuratıon distribution, update collection, weighted averaging of updates, and evaluation of the global model.

T2.4 (w4) FedProx and partial participation. The proximal term will be added to the local objective and each round is using a subset of the registered clients.

T2.5 (w3-w5) Automated tests. Automated tests checks aggregation with small known examples, FedProx, partial participation, and reproducibility with fixed random seeds.

T2.6 (w4-w5) Baselines and early results. Run centralized, local-only, and federated training on the tabular dataset, and report appropriate classification metrics (for example accuracy, F1, and AUC).

T2.7 (w4-w5) Model tuning. Tune the model architecture and hyperparameters (learning rate, number of local epochs, FedProx coefficient), because training on the banking data may be difficult.

Deliverables

D2.1 Simulation framework with the transport interface, an in-memory implementation, and a run script.

D2.2 A full FedAvg run on the tabular banking dataset, shown at the second progress meeting.

D2.3 FedProx and partial participation: both algorithms complete full runs

D2.4 Automated tests for aggregation, FedProx, and partial participation.

D2.5 Early tabular results with centralized, local-only, and federated baselines.

Milestones

M2.1 FedAvg runs end to end in simulation mode.

M2.2 FedAvg and FedProx both complete full runs, including rounds with partial participation.

M2.3 First baseline comparison is available and the tuned model is chosen.

Work package number	**3**	Start date or starting event:		**Week 3-Week 4**	
Work package title	Secure API gateway				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**2**	2	2	2	

Objectives

- Provide a gateway that is the only entry point for clients and receives their model updates over HTTPS.
- Allow only registered clients to connect, over TLS.
- Store the update weights in file storage and record every accepted update in PostgreSQL.
- Let the FL clients from WP2 use the gateway through an HTTPS implementation of the transport interface.

Description of work

T3.1 (w3) Gateway skeleton and storage. Set up the FastAPI gateway with the endpoints defined in the API specification of WP1 (registration, task polling, model download, update upload). Create tables in PostgreSQL for clients and updates and the file storage for weights.

T3.2 (w3) TLS. Configure certificates for the gateway and refuse plain HTTP.

T3.3 (w4) Client registration. Implement list of clients through an administrator to establish clıent registration.

T3.4 (w4) Update intake. Receive updates and their metadata (run id, round id, client id, update id, model version, checksum) Verify checksum and size Store weights in file storage and Record update in PostgreSQL.

T3.5 (w4) HTTPS transport for clients. Implement the client transport interface of WP2 over HTTPS, so that a simulated client can train and upload through the gateway.

Deliverables

D3.1 Gateway with endpoints, PostgreSQL tables, and file storage, running over TLS.

D3.2 Client registratıon will be done through an approved client list.

D3.3 HTTPS implementation of the client transport interface.

Milestones

M3.1 The gateway runs over TLS and accepts the regıstration of a client on the approved list

M3.2 A registered client trains and uploads an update through the gateway, and the update has a record in PostgreSQL.

M3.3 Unregistered clients and plain HTTP requests are rejected.

Work package number	**4**	Start date or starting event:		**Week 4-Week6**	
Work package title	Orchestration and fault tolerance				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**3**	3	3	3	

Objectives

- Run FL rounds automatically over the real gateway, without manual steps between rounds.
- Keep a round state machine (OPEN, COLLECTING, AGGREGATING, DONE/FAILED) whose state is stored in PostgreSQL.
- Make training continue when clients are slow, drop out, reconnect, or send stale or duplicate updates.
- Make a run resumable after the coordinator is restarted.
- Run the same training and orchestration code on separate machines over the Internet, in addition to local simulation.

Description of work

T4.1 (w4) Coordinator reads from the gateway. Connect the coordinator of WP2 to the update records in PostgreSQL and the weights in file storage, so that aggregation uses real client updates arriving through the gateway.

T4.2 (w4-w5) Round state machine. PostgreSQL Store run, round and state after each transition. Implement the round states and ıts transitions.

T4.3 (w5) Participation rules. Set minimum number of participating clients, round timeout, and the decision to aggregate with clients that arrived or fail the round.

T4.4 (w5) Stragglers, connection failures, and retries. Late clients Temporary connection failures while downloading/uploading Client backoff retries with avoiding the doubling count.

T4.5 (w5) Stale and duplicate updates. Discard updates for a closed round version or an old model version and duplicate updates from the same client for the same round (using the UNIQUE (run_id, round_id, client_id) constraint).

T4.6 (w6) Dropout, reconnection, and resume. Handle a client that drops out before a round starts (it is not selected and the round proceeds with the remaining clients) and a client that drops out during a round (its update is missing at the timeout, and the round is aggregated without it or marked as failed according to the participation rule). Let a dropped client rejoin at a later round, and let a run continue from its stored state after a coordinator restart.

T4.7 (w5-w6) Tests. Write automated tests for each failure case: slow client, client dropout before a round, client dropout during a round, temporary connection failure followed by a successful retry, duplicate update, stale update, reconnection in a later round, and coordinator restart. Run a multi-round training with the real gateway.

T4.8 (w5-w6) Real multi-machine deployment. Deploy the gateway, the coordinator, and several clients on separate machines (university cluster nodes and team members' machines), where each client plays the role of one participating organization, and run the same configuration that runs in simulation.

Deliverables

D4.1 Coordinator that aggregates updates received through the gateway.

D4.2 Round state machine with state stored in PostgreSQL.

D4.3 Participation, timeout, retry, and stale/duplicate update handling.

D4.4 Dropout, reconnection, and resume support.

D4.5 Failure-case tests and a multi-round run over the real gateway, with a short test report.

D4.6 Real multi-machine deployment with deployment instructions.

Milestones

M4.1 The coordinator completes one round with updates received through the gateway.

M4.2 Rounds run automatically with minimum participation and timeouts.

M4.3 A multi-round run survives slow clients, dropouts before and during a round, temporary connection failures, duplicate and stale updates, and a coordinator restart.

M4.4 The first training run completes on separate machines over the Internet and is shown at the mid-demo.

Work package number	**5**	Start date or starting event:		**Week 5 -Week 7**	
Work package title	Client compatibility and conformance				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**3**	3	3	3	

Objectives

- Make sure that clients owned by different parties can join a run only if their model, data format, and preprocessing match the global setup.
- Define a conformance contract (manifest) that each client must fulfill before being accepted into a run.
- Support clients running on team provided versioned code (containers or release bundles) where the clients's own code cannot be made public.
- Report compatibility problems clearly, so that a mismatch is found before training and not after a failed round.

Description of work

T5.1 (w5) Conformance manifest. Define the manifest format, which lists model and parameter shapes, feature names, order, and types, units, encoding and preprocessing versions, label meaning and ranges, and reference the test cases. Version the manifest together with each run.

T5.2 (w5-w6) Client-side conformance check. Provide a checker which that the client runs locally against the manifest, including the reference test cases, and generates a pass/fail report without exposing any raw data.

T5.3 (w6) Gateway and coordinator enforcement. The gateway and coordinator will only allow a client to run once the client has submitted a passing conformance report for the current manifest version then save and store result in PostgreSQL.

T5.4 (w6-w7) Managed-code mode. This could be a versioned container or a release bundle with a fixed interface. Package the client so that a client without access to a public repository can run the same code as well and verıfy version at sign up.

T5.5 (w7) Mismatch tests. Test that clients with wrong feature order, wrong unit, wrong label range, different preprocessing version or wrong model shape will be rejected with an clear reason.

T5.6 (w7) Compatibility status. WP6 web application is to provide compatibility status (passed, failed with reason, version) per client.

Deliverables

D5.1 Specification of manifest of conformity and manifest for the banking dataset

D5.2 Client side conformance checker with referencing test cases.

D5.3 Enforcement of gateway and coordinator, results are stored in PostgreSQL.

D5.4 Managed code client package (container or release bundle) with versıon check.

D5.5 Report on mismatch tests and short test report.

Milestones

M5.1 The manifest is defined and first client pass the check.

M5.2 A run cannot have non-conforming client.

M5.3 A client running the managed-code package finish a run.

Work package number	**6**	Start date or starting event:		**Week 4-Week 12**	
Work package title	Observability and web application				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**9**	9	9	9	

Objectives

- Log each run's important events (rounds, updates, failures, timings) in PostgreSQL in machine readable format.
- Producing human readable outputs from in these records: JSON/CSV exports, Markdown reports, and plots.
- Set up a Next.js web application with a public area (the project web site) and an authenticated internal area for following a run live.
- Through a PostgreSQL connection show a same data in Grafana.
- Establish the project web site ın advance and maintain it through to the end of the semester.

Description of work

T6.1 (w4) Public project site. Building public part of the Next.js application: project description, team, progress updates, linking to the repo, poster, and demo video as they come ın.

T6.2 (w4-w5) Event logging. Define the log schema in PostgreSQL, and write events from on the gateway and the coordinator. For each event the run, round, client, message type (registration, model/configuration download, update upload, heartbeat), timestamp, duration, size, status and a failure

reasons are logged. At the beginning of each run, coordinator logs the run configuration and the versions of software, protocol, model, preprocessing and conformance manifest.

T6.3 (w5) Metric collection. Gather the per round and per client metrics, those are needed for evaluation: RTT, transfer time and size per message type, total bytes, retries, dropouts, training, aggregation and waiting time, global and per client ML metrics, and the local performance of each client on its own data before and after local training in each round.

T6.4 (w5-w6) Authenticated internal area. Add login and the internal pages: live round view, update inbox, logs, and client and compatibility status. The internal area reads from the same PostgreSQL records.

T6.5 (w6) End-of-training report. At the end of every run, automatically generate a machine-readable log (JSON/CSV) and a Markdown report. The report includes: the run ID, timestamps, configuration and all software, protocol, model and preprocessing versions; the dataset allocation summary and number of samples per client; participating clients per round and dropout/failure events; global and per-client ML metrics over time; communication and timing metrics with min/max/mean summaries, including per-message-type transfer times; the number of rounds attempted and completed, with the reason for every failed or skipped round; and the final model location and checksum. The report alone is enough to reconstruct what happened in a run without looking at the console output. The main ML metrics and system metrics are automatically plotted.

T6.6 (w6-w7) Grafana. Connect Grafana to PostgreSQL and create dashboards for round status, durations, and failures.

T6.7 (w7-w12) Web site maintenance. Update the public site every week with progress, add the experiment results, poster, demo video, and final report as they are completed, and fix issues found during use.

Deliverables

D6.1 Public project web site, online

D6.2 Log schema and event and metrıc recording ın the gateway and the coordinator

D6.3 Live round view, update inbox, logs and clıents status through internal area.

D6.4 Automatic end of training reports (JSON/CSV log, Markdown summary, and plots).

D6.5 Grafana dashboards will be connected to PostgreSQL.

D6.6 With all project material final version of the web site will be shown

Milestones

M6.1 The public project web site is online.

M6.2 A running training can be followed live in the internal area.

M6.3 A finished run produces the complete report and plots without manual work, and the run can be reconstructed from the report alone.

M6.4 The web site is complete and contains the final deliverables.

Work package number	**7**	Start date or starting event:		**Week 5-Week 10**	
Work package title	Experiments and evaluation				
Participant number	1	2	3	4	5

Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**6**	6	6	6	

Objectives

- Test with the framework on a tabular dataset and one image/text dataset to demonstrate its applicability to different data types.
- FedAvg vs FedProx. Federated training vs centralized training vs local only training comparısons.
- Test the system in with real clients on dıfferent machines over the Internet and with injected failures.
- Reproducibility of all experiments.

Description of work

T7.1 Second modality pipeline. Prepare an image or text dataset, the client partitions, the model, and the conformance manifest.

T7.2 (w7-w8) Experiment design. Fix scenarıos (dataset, number of clients, participation rate, FedAvg or FedProx, local epochs, IID partition, and optional non-IID partition), make the metrics, and the random seeds. Prepare centralized and local-only baselines.

T7.3 (w8-w9) Runs. Run the scenarios on a different machines with several clients over the Internet, including scenarios with slow or dropped clıents and a restart of the coordinator.

T7.4 (w9-w10) Analysis. Collect the metrics from PostgreSQL, produce tables and plots (model quality, round duration, transfer size, retries, dropouts, and waiting versus computing time), and compare the algorithms and baselines.

T7.5 (w10) Reproducibility. Store configuratıon, scripts and instructions to be able to make rerun every reported result.

Deliverables

D7.1 Image pipeline with confarmance manifest D7.1 Text pıpelıne with conformance manifest

D7.2 Plan of experiments whıch including scenarios, metrics and baselines.

D7.3 Results of all scenarios whıch will be stored in PostgreSQL and exported as JSON/CSV.

D7.4 Evaluation tables and plots with an wrıtten comparison in between FedAvg and FedProx and between federated, centralized, and local training.

D7.5 Reproducibility package (configurations, scripts and instructions)

Milestones

M7.1 The image or text datasets can be trained successfully on within the same framework.

M7.2 All planned scenarios have been executıng on separate machines over the Internet.

M7.3 The evaluation results are ready for usega in the report and the poster.

Work package number	**8**	Start date or starting event:	**Week 6 -Week10**		
Work package title	Final model delivery and privacy extension				
Participant number	1	2	3	4	5

Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**5**	5	5	5	

Objectives

- Transfer trained global model to each registered client as a verifiable artifact to yield a usable model on each client.
- Ensure that the delivered model makes the same predictions as the trained model.
- Add an optional privacy extension, if time allows, which clips and sounds client updates.

Description of work

T8.1 (w6) Model artifact. Package the final model into a versioned artifact with checksum, manifest version and training run.

T8.2 (w6-w7) ONNX export. Export the model to ONNX and verify that the outputs are equal to the PyTorch model on the reference test cases.

T8.3 (w7) Delivery. Install a gateway endpoint for downloading the final model. Client checks the check sum and performs a local prediction check.

T8.4 (w7) Delivery test. Check that a registered client gets the model and produces the reference predictions and doesn't an unregistered client.

T8.5 (w9-w10),Optional,Privacy extension. Add update clipping and noise at client side and test its impact on model quality for various levels of noise. This task is done only if the schedule allows

.

Deliverables

D8.1 ONNX export of versioned and check summed model artifact compared with PyTorch.

D8.2 Model download endpoint and verification on client.

D8.3 Delivery tests and a short test report.

D8.4 (optional) Privacy extension with a measurement of its effect on model quality.

Milestones

M8.1 The final model is downloaded by the registered client and the reference predictions are reproduced.

M8.2 (optional) The privacy extension runs and its effect on model quality is measured

Work package number	**9**	Start date or starting event:		**Week 8 -Week 12**	
Work package title	Poster, demo video, and final report				
Participant number	1	2	3	4	5
Participant name	Bora Toker	Enes Tunç	Hilal Aygül	Merve Köybaşı	
Weeks per participant	**5**	5	5	5	

Objectives

- Project clearly in the poster, the demo video, and the final report.
- Show the complete working product.
- Repository and the project web site that others can understand and run.

Description of work

T9.1 (w8-w12) Final report. Write the report step by step during the semester, starting with the finished design and implementation chapters, and add the evaluation results when WP7 finishes.

T9.2 (w9-w10) Poster. Prepare the poster with the problem, the architecture, the main results, and a picture of the web application.

T9.3 (w10-w11) Demo video. Prepare a script and record an end-to-end demonstration: client registration, an automatically orchestrated run, a failure and recovery, the live view and the report, and final model delivery.

T9.4 (w11-w12) Presentation preparation. Prepare and rehearse the poster presentation and the live demo, and prepare a fallback recording in case the live demo fails.

T9.5 (w12) Final check. Clean the repository (README, setup and run instructions, licenses), and check the web site and all links.

Deliverables

D9.1 Final report.

D9.2 Poster.

D9.3 Demo video.

D9.4 Clean repository with setup and run instructions.

Milestones

M9.1 The poster is finished and submitted.

M9.2 The demo video is recorded and published on the project web site.

M9.3 The final report is submitted, and the repository and web site are final.

2.3 Demonstration

Performance analysis measures

All measures are taken from the PostgreSQL log, meaning that there is a traceability of each result to the runs.

Model quality: Both accuracy and F1 score as well as AUC values on a held out test set for the global model and for each client's data. We conduct comparisons between FedAvg and FedProx, between federated training and centralized and local-only training.We also report local performance, before and after local training, for each client.

System performance, Round trip time: Transfer time and size by each message type (model download, update upload, register); Total bytes for each round; Retry, drop out and failed/skipped rounds with reasons; Training, aggregation, waiting and total round duration (min, max, mean).

Robustness: Percentage of rounds ending correctly with slow clients, dropouts before and during rounds, temporary connection failures, duplicate and stale updates, and if a run resumes from a saved run.

Security and correctness: Unregistered clients, plain HTTP, tampered checksums, and mismatching clients are rejected. There is one record in PostgreSQL for every accepted update.

Usability: A complete run should require no manual step between rounds; the report, exports and plots will be generated automatically.

There are no predetermined accuracy goals as the quality that can be achieved is dependent on the data set we will be given.

Final demonstration

The full product will be demonstrated, using a number of clients with distinct machines which can be provided by university cluster nodes and our own machines which will be connected to each other via the Internet, and the demonstration will be recorded as the demo video. A client not registered on the system is rejected, a client registered on the system is connected. If the client's feature order is not equal to the conformance manifest, then the client is rejected with a clear reason. A federated run is initiated, followed by rounds, updates, logs and client status in real time through the web application. We make one client slow down, take another offline for a couple of seconds and let it reconnect so it is able to rejoin the game, and take a third client offline during a round and start the coordinator again. The run is ongoing, and the report displays the event and the cause. A report, plots and Grafana dashboards are created at the end of the run, along with the FedAvg and FedProx comparison from the experiments. Finally a registered client will download the final model, check the model's checksum, and regenerate reference predictions.

2.4 Impact

Technical impact: Numerous federated learning tools focus on simulating federated learning with just a single machine. The missing layers in the above that we introduce to our project are: Encrypted communication, orchestration with failures, inter-organization compatibility checks, observability, and delivery of the final model. It provides a re-usable reference implementation that demonstrates how a simulated FL experiment can be realized as a real system.

Practical impact: The framework can be used by organizations that cannot share raw data, such as banks, to train a joint model while the data remains on their own premises. Each organization shares access to a model trained on more data than they have to train on individually, and can see and verify what is going on while the model is being trained via the live view, the logs and the reports.

Privacy and regulatory impacts: By keeping raw data local, the risk of data breaches is reduced, and it becomes easier to comply with data protection laws such as KVKK and GDPR. The framework provides a foundation for additional privacy efforts and the optional clipping and noise extension is a first step.We dont guarantee formal privacy.

Educational impact: The team experiences hands-on machine learning, secure web services, distributed systems and monitoring. The documented design, the open reports and reproducible experiments may serve as valuable resources to other students and researchers who wish to learn how FL works in other contexts than simulation.

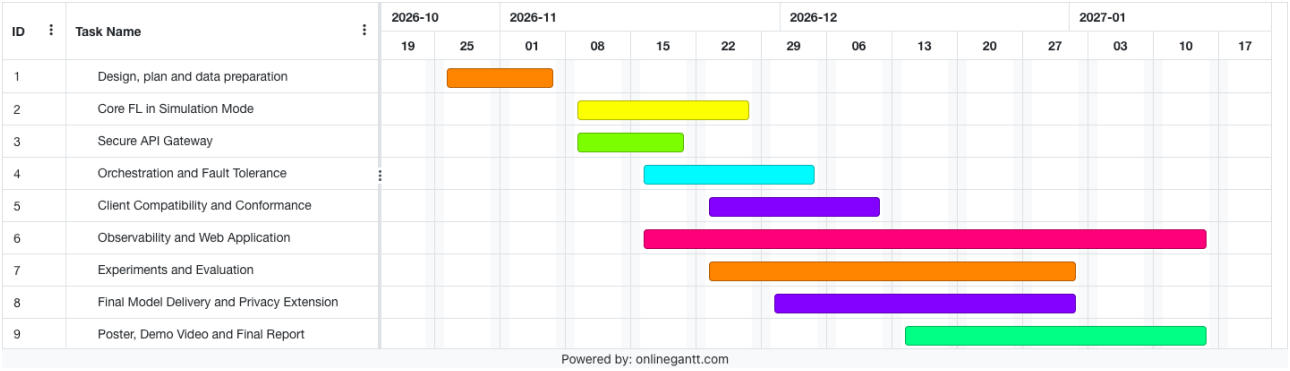
Limits: The framework represents a prototype of a handful of trusted organisations (cross-silo). It is not intended for thousands of devices, and is not a certified production system.

2.5 Risk analysis

Data: The banking data is supplied by the advisor and the exact size, content, quality and license of the data are confirmed during the 1st week. Training a model based on such data can also be difficult. This framework is not reliant on a particular dataset, therefore we use an open tabular dataset for development and testing, and tune the model at an early stage. If the banking data does not arrive on time or cannot be used, the main results are reported on the Open dataset. Holding the data outside the repository.

Learning results: Federated training could take longer to converge than centralized training, particularly if the data of the clients vary. The original experiments indicate that many local epochs can cause slowdowns or instabilities in FedAvg and FedProx was proposed in this context [1] [2]. We begin with an IID split and fine-tune local epochs and learning rate early, and to compare with centralized learning and local-only learning baselines. A weak result is reported honestly with the likely reasons for the result.

2.6 Gantt Chart



Section 3 Economical and Ethical Issues

Economic dimension and constraints: The project will only be applicable to a small number of trusted organizations, as the project is a prototype of four students in one semester. There is no need for software or hardware, we develop it with free open source tools (PyTorch, FastAPI, PostgreSQL, Next.js, Grafana) and run it on our own machines. There is no need to buy additional infrastructure as the participating organizations are simulated with clients in the machines of the team members and, if available, the machines of the University. The main cost of the project is the time of the team. The economic benefit of the outcome is that organizations can train a joint model without having to transport their data to a centralized location, thus eliminating the expense and risk of having to collect and store data centrally.

Ethical issues: Ethical issues are important with a project involving machine learning on banking data and this project is no exception. This is regarding data protection and privacy. We don't guarantee privacy, but we do protect the communication with authentication and encryption, and federated learning leaves raw data on the client side, which means that some information about it can be revealed in the model updates. The second concern is fairness – the same model that performs well on average, can perform poorly for one organisation, so we measure quality at a client level as well as at a local level. Misuse and honesty are the third concern. This is a prototype and should be treated as such; all results are reported, even if they are weak, and all licenses of the tools and papers used in the

construction are respected. We will only use the data as authorised by the advisor and abide by the rules concerning the personal data (KVKK and GDPR) if there is personal data in the data set, otherwise we will exclude the data set from the repository..

Section 4 References

[1] H. B. McMahan, E. Moore, D. Ramage, S. Hampson, and B. Agüera y Arcas, "Communication-Efficient Learning of Deep Networks from Decentralized Data," in *Proc. 20th Int. Conf. on Artificial Intelligence and Statistics (AISTATS)*, 2017. arXiv:1602.05629
[2] T. Li, A. K. Sahu, M. Zaheer, M. Sanjabi, A. Talwalkar, and V. Smith, "Federated Optimization in Heterogeneous Networks," in *Proc. 3rd MLSys Conf.*, 2020. arXiv:1812.06127
[3] P. Kairouz et al., "Advances and Open Problems in Federated Learning," *Foundations and Trends in Machine Learning*, vol. 14, no. 1-2, 2021. arXiv:1912.04977
[4] N. Rieke et al., "The Future of Digital Health with Federated Learning," *npj Digital Medicine*, 2020. arXiv:2003.08119
[5] D. J. Beutel et al., "Flower: A Friendly Federated Learning Research Framework," arXiv:2007.14390, 2020. arXiv
[6] H. R. Roth et al., "NVIDIA FLARE: Federated Learning from Simulation to Real-World," arXiv:2210.13291, 2022. arXiv
[7] C. He et al., "FedML: A Research Library and Benchmark for Federated Machine Learning," arXiv:2007.13518, 2020. arXiv
[8] TensorFlow Federated, https://www.tensorflow.org/federated.
[9] European Parliament and Council, Regulation (EU) 2016/679 (General Data Protection Regulation), Article 5, 2016. Text
[10] Kişisel Verilerin Korunması Kanunu, Law No. 6698, Article 4, 2016. Text
[11] IBM, "Cost of a Data Breach Report 2026," conducted by Ponemon Institute. IBM page